

LabSO 2026

Laboratorio Sistemi Operativi - A.A. 2025-2026

Luigi Dell'Eva - luigi.delleva@unitn.it

System Auditing

Che cos'è il system auditing

L'auditing del sistema monitora gli eventi tramite:



Raccolta log di sistema

Archiviazione persistente delle attività di sistema.



System Call

Monitoraggio in tempo reale delle chiamate al kernel.



Analisi Eventi

Interpretazione dei dati per rilevare anomalie.

Linux integra il **Linux Auditing System**, un insieme di strumenti volti all'auditing di Linux. Oltre ai tool dedicati, il sistema offre **builtins** nativi e programmi esterni per il tracciamento delle system call.

strace e ltrace

Strumenti da shell per il monitoraggio delle **system calls**. **ltrace** estende il monitoraggio alle chiamate di libreria.

Questi programmi si basano sulla system call **ptrace**

```
#include <sys/ptrace.h>
long ptrace(enum __ptrace_request request, pid_t pid, void *addr, void *data);
```

Consente a un processo (tracer) di osservare e controllare l'esecuzione di un altro processo (tracee):

- Monitoraggio delle richieste ed interazioni con il sistema.
- Analisi e modifica della memoria e dei registri del processo specificato.

strace

```
$ strace -p <PID>
```

Monitora tutte le system call di un processo già esistente

```
int main(void){ //open.c
    sleep(10); //Wait so that we can attach to the process
    int fd = open("/tmp/out.txt",O_CREAT|O_RDWR,S_IRUSR|S_IWUSR);
    write(fd,"ciao",4);
    exit(0);
}
```

```
$ ./a.out &
$ strace -p <pi>
```

strace

```
$ strace <binary>
```

Monitora tutte le system call del processo creato tramite l'esecuzione del binario specificato

```
printf("This is a printf from %d\n",getpid());
```

```
$ strace ./a.out
```

Filtrare le chiamate

Se usiamo il comando “`strace -e trace=<filter> -p <PID>`” possiamo filtrare le system call che intendiamo monitorare. Alcune opzioni per il filtro sono:

- `%clock` syscall che leggono o modificano gli orologi di sistema
- `%creds` syscall che leggono o modificano gli ID di utenti e gruppi o i permessi
- `%desc` syscall che utilizzano file descriptors
- `%file` syscalls che usano nomi di file
- `%ipc` syscall che usano IPC
- `%process` syscall per la gestione dei processi
- `%pure` syscall che hanno sempre successo (senza argomento)
- `%signal` syscall relative ai segnali
- `<nome>` syscall con quel nome
- ... <https://strace.io/> per altre info

Esempio

```
//filter.c
printf("This is a printf from %d\n",getpid());
int fd = open("/tmp/out.txt",O_CREAT | O_RDWR,S_IRUSR|S_IWUSR);
write(fd,"ciao",4);
pid_t pid = getpid();
exit(0);
```

```
$ strace -e trace=%file filter.out > /dev/null
$ strace -e trace=%pure filter.out > /dev/null
$ strace -e trace=write,getpid filter.out > /dev/null
```

Contare le chiamate e gli errori

Esiste anche l'opzione “strace -c” per tenere il conteggio delle varie chiamate e degli errori generati.

```
int fd = open("/tmp/out1.txt", O_CREAT | O_RDONLY, S_IRUSR | S_IWUSR);  
int fd2 = open("/tmp/out2.txt", O_CREAT | O_RDWR, S_IRUSR | S_IWUSR);  
write(fd, "ciao", 4);  
write(fd2, "ciao", 4);
```

```
$ strace -e trace=openat,write -c count.out > /dev/null
```

Strace di strace

Che succede se facciamo strace di strace?

```
int fd = open("/tmp/out1.txt", O_CREAT | O_RDONLY, S_IRUSR | S_IWUSR);  
int fd2 = open("/tmp/out2.txt", O_CREAT | O_RDWR, S_IRUSR | S_IWUSR);  
write(fd, "ciao", 4);  
write(fd2, "ciao", 4);
```

```
$ strace strace -e trace=openat,write -c count.out > /dev/null
```

Audit

Audit è un programma fondamentale integrato in molte distribuzioni Linux per la supervisione avanzata del sistema.



Tracciamento

Monitoraggio granulare delle **system calls** per un controllo profondo delle attività.



Monitoraggio FS

Supervisione costante del **filesystem** e degli eventi critici di sistema.



Logging Attivo

Opera come un **demone** sempre attivo che intercetta e logga ogni evento configurato.

Un sistema basato sulle regole

Similmente a molti tool usati in ambito di sicurezza, audit si basa su delle regole che definiscono il tipo di eventi che si vogliono intercettare e loggare. Di default, audit registra una grande quantità di eventi relativi alla sicurezza del sistema.

Per aggiungere una regola si può usare il comando **auditctl <rule>**, mentre per visualizzare le regole attive si può usare **auditctl -l**

*Per renderle persistenti, le regole vanno aggiunte a “/etc/audit/audit.rules”

Esempi di regole

Esistono vari tipologie di regole, quelle più comuni sono per il tracciamento dei file e delle system calls.

Files: `auditctl -w /path/to/fileOrDir -p rwx -k <key>`

- **-w** è seguito dal nome del file o cartella da monitorare
- **-p** è seguito dal tipo di eventi che si vogliono monitorare (**r**ead, **w**rite, **e**xecute, **a**tttribute changes)
- **-k** è seguito da una chiave usata per la ricerca nei logs

Nella sua sintassi, la regola diventerebbe:

```
auditctl -a always,exit -F path=/path/to/fileOrDir -F perm=rwx -F key=<key>
```

Cercare eventi nel log

I log mantenuti da auditd sono generalmente salvati in `/var/log/audit/`.

Tuttavia, auditd è molto prolifico nella generazione di logs, ragion per cui è consigliabile l'utilizzo di **ausearch**. Questo tool accetta diversi parametri che gli consentono di filtrare il contenuto del log in base alle systemcalls, ai files o alle chiavi associate alle regole. Alcuni parametri per filtrare sono:

- **-k** <key> filtra per chiave della regola
- **-p** <PID> filtra per processo
- **-sc** <syscall> filtra per syscall
- **-f** <filename> filtra per file

Esempio: `ausearch -k <key>`